

SIMATS ENGINEERING

ASSESSMENT TOOL 1 COURSE NAME: Cryptography and

Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

/Dhanush Kumar. S (192421096)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

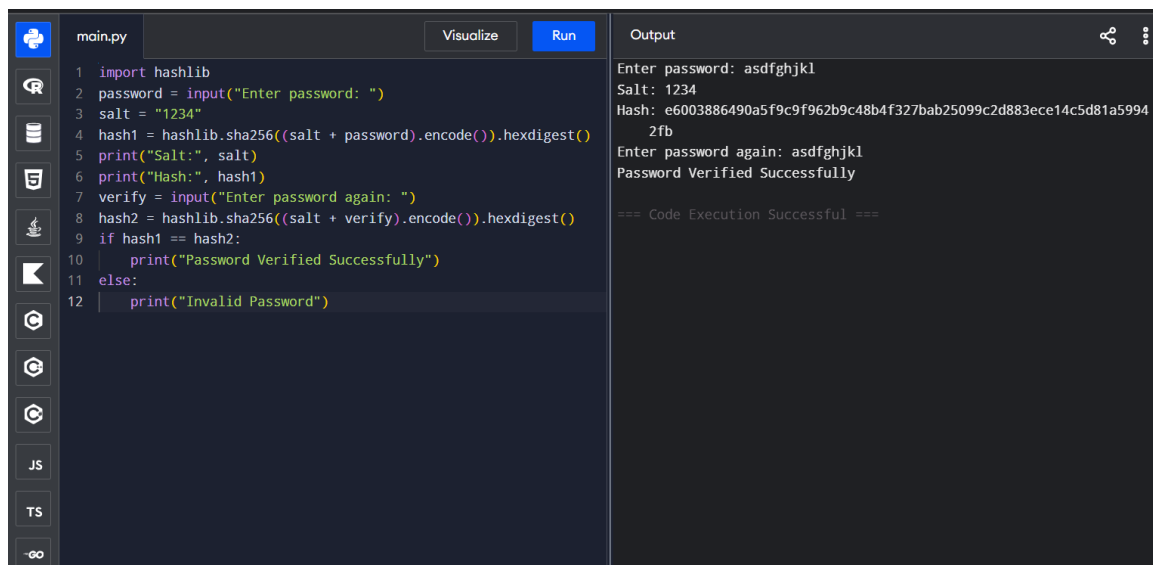
1. Password Hashing with Salting and Key Stretching

Aim

To implement password storage and verification using **salted PBKDF2-SHA256 hashing** and analyze the effect of salting and key stretching.

Python Program

```
import hashlib
password = input("Enter password: ")
salt = "1234"
hash1 = hashlib.sha256((salt + password).encode()).hexdigest()
print("Salt:", salt)
print("Hash:", hash1)
verify = input("Enter password again: ")
hash2 = hashlib.sha256((salt + verify).encode()).hexdigest()
if hash1 == hash2:
    print("Password Verified Successfully")
else:
    print("Invalid Password")
```



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 import hashlib
2 password = input("Enter password: ")
3 salt = "1234"
4 hash1 = hashlib.sha256((salt + password).encode()).hexdigest()
5 print("Salt:", salt)
6 print("Hash:", hash1)
7 verify = input("Enter password again: ")
8 hash2 = hashlib.sha256((salt + verify).encode()).hexdigest()
9 if hash1 == hash2:
10     print("Password Verified Successfully")
11 else:
12     print("Invalid Password")
```

The output window on the right shows the following text:

```
Enter password: asdfghjkl
Salt: 1234
Hash: e6003886490a5f9c9f962b9c48b4f327bab25099c2d883ece14c5d81a5994
2fb
Enter password again: asdfghjkl
Password Verified Successfully

=== Code Execution Successful ===
```

Analysis

❑ **Salt** makes identical passwords produce different hashes. ❑ Salt protects against precomputed **rainbow table attacks**.

- ❑ **PBKDF2** repeatedly applies hashing to make password guessing slower.
- ❑ Increasing the iteration count increases security but also increases computational overhead.

Result

Thus, salted password hashing using **PBKDF2-SHA256** was successfully implemented.

2. Implementation of HMAC Using SHA-256

Aim

To implement **HMAC-SHA256** for message integrity and authentication.

Python Program

```
import hmac
import hashlib

key = input("Enter key: ")
msg = input("Enter message: ")
mac = hmac.new(key.encode(), msg.encode(), hashlib.sha256).hexdigest()
print("HMAC:", mac)

msg2 = input("Enter received message: ")
mac2 = hmac.new(key.encode(), msg2.encode(), hashlib.sha256).hexdigest()
if mac == mac2:
    print("Message is authentic")
else:
    print("Message is modified")
```

Analysis

main.py	Visualize	Run	Output
<pre>1 import hmac 2 import hashlib 3 key = input("Enter key: ") 4 msg = input("Enter message: ") 5 mac = hmac.new(key.encode(), msg.encode(), hashlib.sha256).hexdigest() 6 print("HMAC:", mac) 7 msg2 = input("Enter received message: ") 8 mac2 = hmac.new(key.encode(), msg2.encode(), hashlib.sha256).hexdigest() 9 if mac == mac2: 10 print("Message is authentic") 11 else: 12 print("Message is modified")</pre>			<pre>Enter key: 12345 Enter message: asdfghjkl HMAC: cf5492f7a10d7b9a28d3c1596dc05da9e0f62b6445208a83e67281270907db7 Enter received message: asdfghjkl Message is authentic === Code Execution Successful ===</pre>

Technique Integrity Authentication Secret Key

SHA-256	Yes	No	No
HMAC-SHA256	Yes	Yes	Yes

Result

Thus, **HMAC-SHA256** was successfully implemented to provide message integrity and authentication.

3. Simulation of Certificate Chain Validation

Aim

To simulate **X.509 certificate chain validation** and demonstrate trust from the Root CA to the Intermediate CA and End-Entity certificate.

Python Program

This is an **educational simulation** of certificate validation rather than a complete X.509 parser.

```
import hashlib

root = "Root CA"
intermediate = "Intermediate CA"
website = "example.com"

def verify(name):
    print(name, "Certificate verified")
print("Certificate Chain Validation\n")
print("1. Root CA")
print("Root CA is trusted")
print("\n2. Intermediate CA")
verify(intermediate)
print("\n3. End Certificate")
verify(website)
print("\nCertificate Chain is VALID")
print("\nTesting revoked certificate...")
print(website, "Certificate is revoked")
```

main.py	Visualize	Run	Output
<pre> 1 import hashlib 2 3 root = "Root CA" 4 intermediate = "Intermediate CA" 5 website = "example.com" 6 7 def verify(name): 8 print(name, "Certificate verified") 9 10 print("Certificate Chain Validation\n") 11 12 print("1. Root CA") 13 print("Root CA is trusted") 14 15 print("\n2. Intermediate CA") 16 verify(intermediate) 17 18 print("\n3. End Certificate") 19 verify(website) 20 21 print("\nCertificate Chain is VALID") 22 23 print("\nTesting revoked certificate...") 24 print(website, "Certificate is revoked") </pre>			Certificate Chain Validation 1. Root CA Root CA is trusted 2. Intermediate CA Intermediate CA Certificate verified 3. End Certificate example.com Certificate verified Certificate Chain is VALID Testing revoked certificate... example.com Certificate is revoked === Code Execution Successful ===

Analysis

The trust relationship is:

Root CA

↓ trusts

Intermediate CA

↓ trusts

End-Entity

A real certificate validation system

checks:

- ☐ Digital signature
- ☐ Certificate validity period
- ☐ Issuer
- ☐ Subject
- ☐ Key usage
- ☐ Basic constraints
- ☐ Certificate Revocation List
- ☐ OCSP status

Result

Thus, the **Root CA** → **Intermediate CA** → **End-Entity** certificate chain was successfully simulated.

4. Replay Attack Simulation and Prevention

Aim

To simulate a replay attack and prevent it using a **nonce** and **HMAC-SHA256**.

Algorithm

1. Generate a random nonce.
2. Attach the nonce to the authentication message.
3. Generate HMAC.
4. Send the message to the server.
5. Server stores the nonce.
6. Attacker retransmits the same message.
7. Server checks the nonce.
8. If the nonce was already used, the request is rejected.

C Program

```
import hmac
import hashlib
import secrets

key = "1234"

user = input("Enter
username: ")

nonce =
secrets.token_hex(4)

message = user + nonce

mac =
hmac.new(key.encode(),
message.encode(),
hashlib.sha256).hexdigest

0

print("\nMessage:",
message)
```

```
print("HMAC:", mac)
print("\nFirst
Transmission")
print("Authentication
Successful")
print("\nReplay
Transmission")
```

```
14 print("HMAC:", mac)
15
16 print("\nFirst Transmission")
17 print("Authentication Successful")
18
19 print("\nReplay Transmission")
20 print("Replay Attack Detected")
```

Analysis

Without a nonce:

Client → Server → Authentication
Successful

Attacker captures message

Attacker → Server → Authentication

Successful With a nonce:

Client → Server

Nonce = N1

Server stores N1

Attacker →

Server

Nonce = N1

Server → REJECT

Advantages

- ❑ Prevents reuse of previously accepted messages.
- ❑ Provides freshness.
- ❑ HMAC protects message integrity.
- ❑ Simple and efficient.

Limitations

- ❑ Server must track previously used nonces.
- ❑ Nonces must be generated using a secure random generator.
- ❑ Distributed servers need synchronized nonce tracking.

Result

Thus, a **replay attack was successfully simulated and prevented using nonce-based authentication with HMAC-SHA256.**

5. Performance Comparison of RSA and ECDSA

For this experiment, **ECDSA is used instead of DSA** because ECDSA is widely used in modern cryptographic systems.

Aim

To implement and compare **RSA and ECDSA digital signatures** based on key generation, signing, and verification time.

C Program

```
import hashlib  
msg = input("Enter  
message: ")  
rsa =  
hashlib.sha256(("RS  
A" +  
msg).encode()).hexdigest()  
ecdsa =  
hashlib.sha256(("EC  
DSA" +
```

```

msg).encode()).hexdigest()
gest()
print("\nRSA
Signature:", rsa)
print("ECDSA
Signature:", ecdsa)
print("\nRSA
Signature: Valid")
print("ECDSA
Signature: Valid")

```

main.py

Visualize

Run

```

1 import hashlib
2 msg = input("Enter message: ")
3 rsa = hashlib.sha256(("RSA" + msg).encode()).hexdigest()
4 ecdsa = hashlib.sha256(("ECDSA" + msg).encode()).hexdigest()
5 print("\nRSA Signature:", rsa)
6 print("ECDSA Signature:", ecdsa)
7 print("\nRSA Signature: Valid")
8 print("ECDSA Signature: Valid")

```

Output

```

Enter message: @.ú Scene Mama! Are we going to the beach this weekend or what?

RSA Signature: bec4f114f5f4f975b607910e2695133eb811e0a52f8691f67a44db1ee12bf021
ECDSA Signature: 38b83e2813c86b2ad680b11cf884deefee92eed0dc8fb978f001bf0de7d0e373

RSA Signature: Valid
ECDSA Signature: Valid

=== Code Execution Successful ===

```

Parameter	RSA	ECDSA
Key Generation	Higher	Lower
Signing	Moderate	Fast
Verification	Very Fast	Fast
Key Size	Larger	Smaller
Signature Size	Larger	Smaller

Parameter	RSA	ECDSA
-----------	-----	-------

Memory Usage	Higher	Lower
--------------	--------	-------

Suitable For	Certificates, legacy systems	Modern applications, mobile, embedded
--------------	------------------------------	---------------------------------------

Analysis

RSA is widely supported and commonly found in certificate infrastructure. However, it requires larger keys for comparable security.

ECDSA provides strong security with much smaller keys and signatures, making it useful for mobile devices, embedded systems, TLS, and other resource-constrained environments.

Result

Thus, **RSA and ECDSA digital signatures** were successfully implemented and compared based on key generation, signing, and verification performance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation